

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-04 (Unidad 2 - Tarea 2)
Implementación de la estructura completa de una base de datos (DDL)
en tres (3) SGBDs diferentes

- **Tarea en Equipo**
- **Peso: 15%**
- **Práctica. Caso de Estudio**
- **Modelo Físico - Diccionario de Datos en DBMS - PostgreSQL, MySQL, MS SQL Server**
- **Datos Estructurados y Semi Estructurados**

MIEMBROS DEL EQUIPO:

- Líder: SARA PALACIO ZAPATA
- Miembro: JULIÁN VELÁSQUEZ SALAS

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto de red social pascualina, ya logró mejorar los modelos conceptual y lógico; y en base al resultado, construyó un modelo físico en un SGBD en específico.

En este contexto, el reto consiste en tomar el modelo lógico y físico (en un SGBD específico) previamente construido y completamente funcional, e implementarlo en otros dos (2) Sistemas de Gestión de Bases de Datos (SGBD), aplicando rigurosamente las reglas del Lenguaje de Definición de Datos (DDL).

Es importante la identificación correcta de los tipos de datos de los SGBDs en los que implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico en los tres (3) SGBDs requeridos, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos genéricos en los tipos de datos del SGBD.

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo
- Justifiquen mejoras
- Transformen correctamente el modelo lógico en físico
- Implementen la base de datos en tres SGBD distintos:
 - PostgreSQL
 - MySQL
 - Microsoft SQL Server

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3. Instrucciones de realización de la tarea

Experimentar el SGBD establecido, de acuerdo con las necesidades de la red social. Luego, utilizando una herramienta gráfica, como pgAdmin4, MySQL Workbench y MS SQL Server Studio, se enfocarán en la correcta implementación del modelo, prestando especial atención a la identificación de tablas, campos, los tipos y tamaños de datos, y, fundamentalmente, la definición de las claves primarias y las relaciones foráneas que dan coherencia al sistema. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final.

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables claves:

- Un informe técnico que justifique sus decisiones.
- Un cuadro comparativo de Tipos de Datos de los diferentes SGBDs
- Inventario de Tablas (resultado de la tarea anterior)
- Diccionario de Datos Físico para cada uno de los 3 SGBDs
- El conjunto de scripts con los modelos físicos (DDL) de los diferentes SGBDs.
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal. **DEBE MOSTRARSE EL ESTUDIANTE E IDENTIFICARSE CON SU NOMBRE. DEBE MOSTRAR CÓDIGO EN EJECUCIÓN -> Mostrar ejecución de scripts de creación y modificación.**

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DDL y la implementación de una base de datos en tres (3) SGBD diferentes a partir del mismo Diccionario Lógico, habilidades fundamentales para cualquier desarrollador de software. Adicionalmente, tendrá la oportunidad de poder comparar la implementación en diferentes ambientes y la utilización de diversos tipos de datos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

3. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura:
Carpeta raíz: Tarea4. En esta carpeta estarán dos carpetas con los entregables: Informe, Resultados y Video.

Carpetas	Contenido requerido	Observaciones
/tarea4	Todas las subcarpetas y archivos de la tarea. Nota: USO OBLIGATORIO DE TODAS LAS PLANTILLAS SUMINISTRADAS	No aplica
tarea4/Informe	Informe detallado	.pdf
tarea4/Diccionarios	Diccionario de Datos Físico (plantilla suministrada) • Tabla comparativa de Tipos de Dato • Inventario de Tablas • Diccionarios de Datos	.xlsx
tarea4/Scripts	Script de creación de la base de datos para PostgreSQL Script de creación de la base de datos para MySQL Script de creación de la base de datos para MS SQL Server Script de modificación de la base de datos PostgreSQL para incluir datos estructurados y semi estructurados	.sql
tarea4/Video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y las conclusiones individuales.	Enlace a YouTube

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: • Identificación de los tipos de datos de los diferentes SGBD • Descripción de cada tabla implementada con: nombre de la tabla, listado de campos con su tipo de dato y tamaño (si aplica), justificación de la elección de tipos de datos para campos clave (por ejemplo, por qué eligieron VARCHAR en lugar de TEXT para un campo específico, o INT en lugar de BIGINT), identificación de las claves primarias (PK yFK) y justificación de su elección. • Descripción de cómo se implementaron las relaciones entre tablas. Para cada relación incluir: tablas involucradas (tabla padre, tabla hija y/o tabla pivote), campos que conforman la clave foránea, especificación de las acciones referenciales (ON DELETE, ON UPDATE, ON CASCADE) y su justificación. • El informe debe referenciar los scripts DDL y explicar su estructura. • Utilización de datos Semi Estructurados (JSON y/o JSONB) • Conclusiones individuales y grupales individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso. • Participación en Foros de Tareas	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Evidencia	Habilidad evaluada
Tipos de dato SQL y NoSQL para big data e IoT con justificación técnica del uso de tipos de datos adecuados para escenarios de big data e IoT Ejemplo: campo JSON para las coordenadas geográficas u optimización del tamaño del dato para captar señales de sensores en IoT.	Comprensión y utilización de tipos de datos que representen una relación con soluciones big data e IoT.
Modelo físico con: • Inventario completo de tablas de la base de datos. • Tablas con nombres según las reglas y la nomenclatura. • Campos correctamente identificados con tamaño y tipo asociado. • Relaciones adecuadas al caso (claves primarias y foráneas). • Uso correcto las reglas, nomenclatura y estándares para BD físicas. • Utilización de datos tipo JSON o JSONB en alguna Tabla PostgreSQL	Comprensión del modelo físico
Video de sustentación (presentarse con imagen y nombre. Mostrar y explicar código en ejecución)	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

Analicen los criterios de evaluación y verifiquen su cumplimiento..

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Informe con resultados

1.- Tabla Comparativa de tipos de Datos

La siguiente tabla presenta la equivalencia entre los tipos de datos genéricos del Diccionario Lógico y los tipos nativos de cada uno de los tres SGBDs implementados.

Tipos de Datos - Tabla Comparativa					
#	Tipo de Dato Genérico	Descripción	En PostgreSQL	En MySQL	En MS SQL Server
			Versión: 18.1	Versión: 8.0	Versión: 2019+
1	AUTOINCREMENTABLE	Número entero que incrementa automáticamente. Usado como PK.	SERIAL	INT AUTO_INCREMENT	INT IDENTITY(1,1)
2	UUID	Identificador único universal de 128 bits.	UUID	CHAR(36)	UNIQUEIDENTIFIER
3	ENTERO	Número entero sin decimales.	INT / SMALLINT / BIGINT	INT / SMALLINT / BIGINT	INT / SMALLINT / BIGINT
4	DECIMAL	Número con parte decimal y precisión exacta.	NUMERIC(p,s)	DECIMAL(p,s)	DECIMAL(p,s)
5	LÓGICO	Valor booleano verdadero/falso.	BOOLEAN	TINYINT(1)	BIT
6	TEXTO	Cadena de caracteres de longitud variable ilimitada.	TEXT	TEXT / LONGTEXT	VARCHAR(MAX)
7	CARACTER	Cadena de caracteres de longitud variable con límite.	VARCHAR(n)	VARCHAR(n)	VARCHAR(n)
8	FECHA	Fecha sin componente horario.	DATE	DATE	DATE
9	FECHA_HORA	Fecha y hora combinadas.	TIMESTAMP	DATETIME	DATETIME
10	HORA	Solo componente de tiempo.	TIME	TIME	TIME
11	URL	Cadena de texto para almacenar URLs.	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)
12	ENUM	Lista de valores predefinidos y controlados.	VARCHAR(n) + CHECK	ENUM('v1','v2',...)	VARCHAR(n) + CHECK
13	TEXTO_CORTO	Cadena de longitud fija.	CHAR(n)	CHAR(n)	CHAR(n)
14	BINARIO	Datos binarios en crudo.	BYTEA	BLOB / LONGBLOB	VARBINARY(MAX)
15	JSON	Datos semiestructurados en formato JSON (texto).	JSON	JSON	NVARCHAR(MAX) c/ validación
16	JSONB	JSON binario indexable y consultable.	JSONB	No nativo (JSON)	No nativo
17	IMAGEN	Referencia a imagen (URL) o binario.	VARCHAR(255) / BYTEA	VARCHAR(255) / BLOB	VARCHAR(255) / VARBINARY(MAX)
18	AUDIO	Referencia a archivo de audio (URL) o binario.	VARCHAR(255) / BYTEA	VARCHAR(255) / BLOB	VARCHAR(255) / VARBINARY(MAX)

Las decisiones de conversión más relevantes son:

- **AUTOINCREMENTABLE:** PostgreSQL usa SERIAL, MySQL usa INT AUTO_INCREMENT, SQL Server usa INT IDENTITY(1,1). Los tres logran el mismo efecto pero con sintaxis completamente diferente.
- **LÓGICO:** PostgreSQL tiene BOOLEAN nativo. MySQL usa TINYINT(1) donde 0=false y 1=true. SQL Server usa BIT con la misma convención numérica.
- **TEXTO libre:** PostgreSQL y MySQL usan TEXT. SQL Server deprecó TEXT y requiere VARCHAR(MAX), que tiene el mismo comportamiento pero diferente nombre.
- **JSON semiestructurado:** PostgreSQL ofrece JSONB (binario, indexable con GIN) además de JSON. MySQL tiene JSON nativo desde 5.7.8. SQL Server no tiene tipo JSON nativo — se usa NVARCHAR(MAX) con funciones JSON_VALUE() y OPENJSON().
- **FECHA_HORA:** PostgreSQL usa TIMESTAMP. MySQL y SQL Server usan DATETIME. La semántica es equivalente pero la precisión de microsegundos varía.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

- DEFAULT de fecha actual: NOW() en PostgreSQL, CURRENT_TIMESTAMP en MySQL, GETDATE() en SQL Server.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

El inventario es común a los tres SGBDs. Comprende 30 tablas organizadas en grupos funcionales, con sus claves primarias, foráneas y restricciones de unicidad documentadas.

Modelo Lógico - Tablas resultantes del Diccionario Genérico (corregido)			
INVENTARIO DE TABLAS			
Este inventario es el mismo para los 3 SGBDs: PostgreSQL, MySQL y MS SQL Server. Todos los diccionarios deben tomar la misma cantidad y los mismos nombres de esta tablas			
#	Tabla	Descripción Tabla	Observaciones
1	usuario	Supertipo central. Atributos comunes de todos los usuarios registrados en Pascualina.	PK: id_usuario UK: email
2	estudiante	Subtipo de USUARIO. Estudiante activo matriculado en la institución.	PK: id_usuario FK: id_usuario, id_programa
3	docente	Subtipo de USUARIO. Profesor vinculado a la institución.	PK: id_usuario FK: id_usuario
4	empleado	Subtipo de USUARIO. Personal administrativo u operativo.	PK: id_usuario FK: id_usuario
5	egresado	Subtipo de USUARIO. Graduado de la institución.	PK: id_usuario FK: id_usuario, id_programa
6	publico_general	Subtipo de USUARIO. Persona externa que participa en la comunidad.	PK: id_usuario FK: id_usuario
7	programa_academico	Catálogo de programas de formación que ofrece la institución.	PK: id_programa UK: codigo
8	habilidad	Catálogo de competencias. Relación N:M con USUARIO.	PK: id_habilidad UK: nombre
9	interes	Catálogo de intereses para conectar usuarios con afinidades comunes.	PK: id_interes UK: nombre
10	publicacion	Contenido generado por los usuarios. Puede estar asociada a un grupo.	PK: id_publicacion FK: id_usuario, id_grupo
11	publicacion_multimedia	Tabla 1FN. Almacena archivos/URLs adjuntos a publicaciones.	PK: id_multimedia FK: id_publicacion
12	comentario	Respuestas con soporte para hilos anidados mediante FK recursiva id_padre.	PK: id_comentario FK: id_usuario, id_publicacion, id_padre
13	reaccion	Reacciones de usuarios a publicaciones. Máximo una por usuario-publicación.	PK: id_reaccion UK: (id_usuario, id_publicacion)

- Supertipo y subtipos de usuario: usuario, estudiante, docente, empleado, egresado, publico_general.
- Catálogos independientes: programa_academico, habilidad, interes, empresa.
- Relaciones N:M y reflexivas: usuario_habilidad, usuario_interes, seguimiento, miembro_grupo, asistencia_evento, postulacion.
- Contenido social: publicacion, publicacion_multimedia, comentario, reaccion, grupo, evento.
- Marketplace Cambalache Store: producto, producto_foto, intercambio.
- Bolsa de trabajo: oferta_laboral, oferta_requisito.
- Comunicación y gestión: mensaje, notificacion, reporte.

Institución Universitaria Pascual Bravo

Facultad de Ingeniería - Departamento de Sistemas Digitales

Base de Datos I (SD1006)

3. Diccionario de Datos y Scripts de PostgreSQL

3.1 Diccionario de Datos Físico — PostgreSQL

El diccionario físico de PostgreSQL se elaboró en la hoja 'Diccionario Datos - PostgreSQL' del archivo resultados.xlsx. Se mantiene el modelo de la Tarea 3 con la adición de dos campos JSONB: perfil_extendido en usuario y metricas_evento en evento.

PostgreSQL - Tablas en Detalle																									
1	Tabla	usuario	Descripción Tabla	Superítipo central. Atributos comunes de todos los usuarios registrados en Pascualina.										v1.0											
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios																
1	id_usuario	Identificador único del usuario	SERIAL		No	S			PK autoincremental																
2	nombre	Nombre(s) del usuario	VARCHAR(100)	100																					
3	apellido	Apellido(s) del usuario	VARCHAR(100)	100																					
4	email	Correo electrónico del usuario	VARCHAR(150)	150	No			S	Valor único																
5	hash_contraseña	Hash de la contraseña	VARCHAR(255)	255					Nunca texto plano																
6	fecha_registro	Fecha y hora de registro	TIMESTAMP						DEFAULT NOW()																
7	foto_perfil	URL de la foto de perfil	VARCHAR(255)	255																					
8	biografia	Texto de presentación	TEXT	MAX																					
9	estado	Estado de la cuenta	VARCHAR(20)	20					DEFAULT 'activo'; CHECK																
10	perfil_extendido	Datos semiestructurados del perfil (Big Data)	JSONB						Índice GIN en PostgreSQL																
N																									
NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.																									
2	Tabla	estudiante	Descripción Tabla	Subtipo de USUARIO. Estudiante activo matriculado en la institución.										v1.0											
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios																
1	id_usuario	FK al superítipo usuario	INT		No	S	S		PK y FK a usuario																
2	codigo_estudiante	Código estudiantil	VARCHAR(20)	20																					
3	semestre	Semestre actual (1-10)	SMALLINT						CHECK BETWEEN 1 AND 10																
4	id_programa	FK al programa académico	INT				S		FK a programa_academico																
N																									
NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.																									
3	Tabla	docente	Descripción Tabla	Subtipo de USUARIO. Profesor vinculado a la institución.										v1.0											
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios																
1	id_usuario	FK al superítipo usuario	INT		No	S	S		PK y FK a usuario																
2	codigo_docente	Código docente	VARCHAR(20)	20																					

3.2 Script de Creación — PostgreSQL

El script postgres-crea.sql contiene la creación del esquema pascualina y las 30 tablas en orden estricto: independientes primero, dependientes después. Cada tabla incluye PKs, FKs, CHECKs y restricciones UNIQUE con nomenclatura pk_, fk_, ck_, uq_.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

The screenshot displays a PostgreSQL database interface. On the left, the 'pascualina' schema is expanded, showing a list of tables including 'asistencia_evento', 'comentario', 'docente', 'egresado', 'empleado', 'empresa', 'estudiante', 'evento', 'grupo', 'habilidad', 'intercambio', 'interes', 'mensaje', 'miembro_grupo', 'notificacion', 'oferta_laboral', 'oferta_requisito', 'postulacion', 'producto', 'producto_foto', 'programa_academico', 'publicacion', 'publicacion_multimedia', 'publico_general', 'reaccion', 'reporte', 'seguimiento', 'usuario', 'usuario_habilidad', and 'usuario_interes'.

The main window shows a SQL query script with the following content:

```

1  --
2  -- Scripts de Creación de la Base de Datos
3  -- Red Social Estudiantil Pascualina
4  -- Grupo: 11 | Equipo: Sara Palacio Zapata, Julián Velásquez Salas
5  -- SGBD: PostgreSQL
6  --
7  -- Todas las instrucciones se DEBEN EJECUTAR EN SECUENCIA SIN ERRORES
8  -- NOTA: Ojo con las tablas relacionadas. Primero las independientes y des
9  --
10 --
11 -- Crear el esquema principal
12 CREATE SCHEMA IF NOT EXISTS pascualina;
13 SET search_path TO pascualina;
14
15 --
16 -- Creación de las tablas
17 --
18
19 -- =====
20 -- TABLAS INDEPENDIENTES (sin claves foráneas)
21 -- =====
22
23 CREATE TABLE programa_academico (
24     id_programa SERIAL NOT NULL,
25     codigo VARCHAR(20) NOT NULL,
26     nombre VARCHAR(150) NOT NULL,
27     facultad VARCHAR(100) NULL,
28     nivel VARCHAR(50) NULL,
29     CONSTRAINT pk_programa_academico PRIMARY KEY (id_programa),
30     CONSTRAINT uq_programa_codigo UNIQUE (codigo)
31 );
32
33 CREATE TABLE habilidad (
34     id_habilidad SERIAL NOT NULL,
35     nombre VARCHAR(100) NOT NULL,
36     categoria VARCHAR(100) NULL,

```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'.

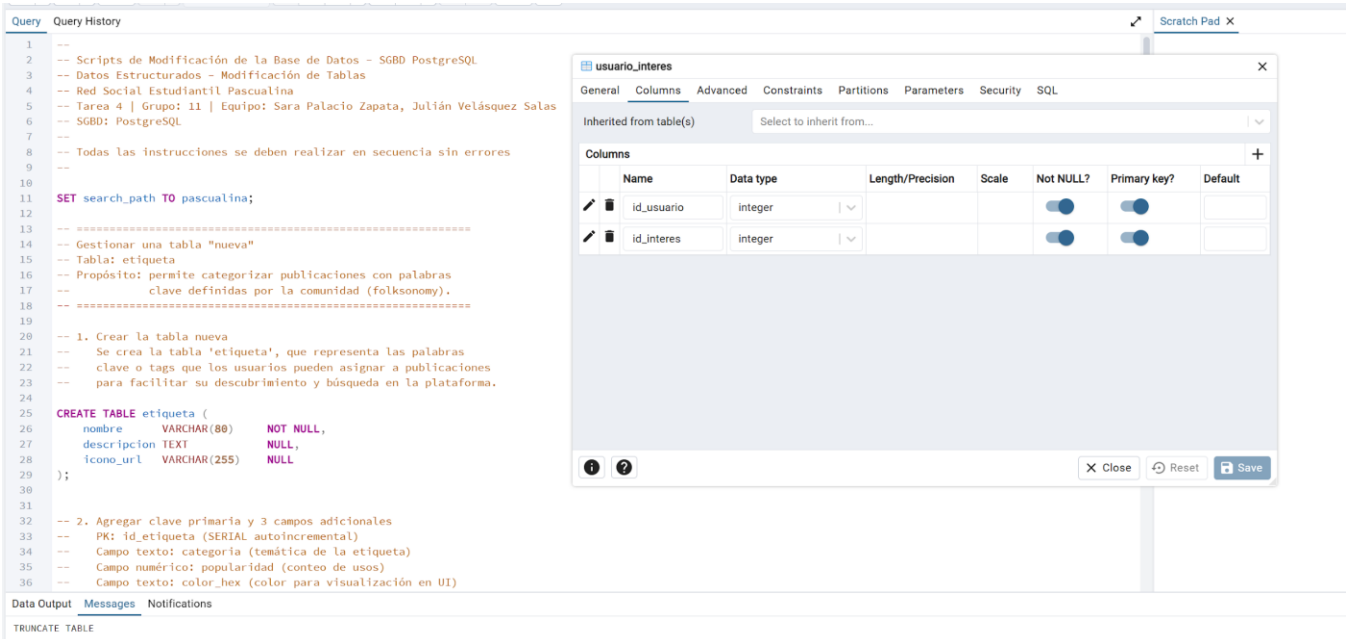
3.3 Script de Modificación — Datos Estructurados

El script `postgre-mod-01.sql` ejecuta 12 operaciones DDL en secuencia sobre la tabla `etiqueta`, renombrada a `tag`. La tabla representa la categorización de publicaciones mediante palabras clave (folksonomy). Los 12 pasos son:

- Crear la tabla `etiqueta` con `nombre`, `descripcion` e `icono_url`.
- Agregar PK (`id_etiqueta`), campo texto `categoria`, campo numérico `popularidad` y campo texto `color_hex`.
- Eliminar el campo `icono_url` (centralización de multimedia).
- Renombrar la tabla de `etiqueta` a `tag`.
- Agregar restricción `UNIQUE` en `nombre`.
- Agregar `fecha_inicio` y `fecha_fin` con `CHECK` de orden entre fechas.
- Agregar campo entero `total_usos` con `CHECK >= 0`.
- Ampliar `categoria` de `VARCHAR(80)` a `VARCHAR(150)`.
- Agregar `CHECK` de rango 0-100000 en `popularidad`.
- Crear índice sobre `categoria` para optimizar búsquedas.
- Eliminar `fecha_fin` — los tags no caducan.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

- TRUNCATE TABLE tag RESTART IDENTITY CASCADE.



3.4 Script de Modificación — Datos Semi-estructurados (JSONB)

El script postgre-mod-02.sql implementa dos campos JSONB sobre tablas existentes, con justificación en Big Data e IoT:

Campo JSONB	Tabla	Caso de uso	Justificación técnica
perfil_extendido	usuario	Big Data — motor de recomendación de mentores	Cada tipo de usuario tiene atributos variables: redes sociales, certificaciones, idiomas, disponibilidad. Un modelo relacional puro requeriría decenas de columnas NULLables. JSONB permite flexibilidad + índice GIN para búsquedas eficientes a escala.
metricas_evento	evento	IoT — sensores en eventos presenciales y analytics de streaming	Los eventos presenciales generan datos de sensores (aforo NFC, temperatura, ruido) y los virtuales generan métricas de streaming heterogéneas. JSONB con GIN permite consultas analíticas sin mover datos a un almacén externo.

Cada campo incluye: ALTER TABLE para agregar el campo JSONB, CREATE INDEX con GIN para consultas eficientes, INSERT/UPDATE de registros de prueba con estructuras JSON reales, y cuatro consultas que demuestran el uso de operadores nativos de PostgreSQL (->, ->>, ?, @>).

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

pascualina/postgres@Local

Query History

```
78
79 -- Consulta 1: Ver perfil extendido completo de todos los usuarios
80 SELECT
81     id_usuario,
82     nombre,
83     apellido,
84     perfil_extendido
85 FROM usuario
86 WHERE perfil_extendido IS NOT NULL;
87
88 -- Consulta 2: Filtrar usuarios con LinkedIn registrado
89 SELECT
90     nombre,
91     apellido,
92     perfil_extendido -> 'redes_sociales' -> 'linkedin' AS linkedin
93 FROM usuario
94 WHERE perfil_extendido ? 'redes_sociales'
95     AND perfil_extendido -> 'redes_sociales' ? 'linkedin';
96
97 -- Consulta 3: Buscar usuarios con disponibilidad en "tardes"
98 SELECT
99     nombre,
100    apellido,
101    perfil_extendido ->> 'disponibilidad' AS disponibilidad
102 FROM usuario
103 WHERE perfil_extendido ->> 'disponibilidad' ILIKE '%tardes%';
104
105 -- Consulta 4: Usuarios interesados en mentoría de "Bases de datos"
106 SELECT
107     nombre,
108     apellido,
109     perfil_extendido -> 'intereses_mentoria' AS intereses
110 FROM usuario
111 WHERE perfil_extendido -> 'intereses_mentoria' ? 'Bases de datos';
112
113
114
```

Data Output Messages Notifications

	nombre character varying (150)	apellido character varying (100)	intereses jsonb
1	Sara	Palacio	["Bases de datos", "Analítica de dat...

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

4. Diccionario de Datos y Scripts de MySQL

4.1 Diccionario de Datos Físico — MySQL

El diccionario físico de MySQL se elaboró en la hoja 'Diccionario Datos - MySQL' del archivo resultados.xlsx, con los tipos de dato nativos de MySQL 8.x. Las diferencias principales respecto a PostgreSQL son TINYINT(1) para booleanos, TEXT/LONGTEXT para texto libre, DATETIME para fecha-hora y JSON para semiestructurados.

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y

MySQL - Tablas en Detalle

1

Tabla

usuario

Descripción Tabla

Superíndice central. Atributos comunes de todos los usuarios registrados en Pascualina.

v1.0

#

Campo

Descripción Campo

Tipo Dato - MySQL

Tamaño

Nulo

PK

FK

UK

Nota/Comentarios

1

id_usuario

Identificador único del usuario

INT AUTO_INCREMENT

No

S

PK autoincremental

2

nombre

Nombre(s) del usuario

VARCHAR(100)

100

3

apellido

Apellido(s) del usuario

VARCHAR(100)

100

4

email

Correo electrónico del usuario

VARCHAR(150)

150

No

S

Valor único

5

hash_contraseña

Hash de la contraseña

VARCHAR(255)

255

Nunca texto plano

6

fecha_registro

Fecha y hora de registro

DATETIME

DEFAULT NOW()

7

foto_perfil

URL de la foto de perfil

VARCHAR(255)

255

8

biografía

Texto de presentación

TEXT

MAX

9

estado

Estado de la cuenta

VARCHAR(20)

20

DEFAULT 'activo'; CHECK

10

perfil_extendido

Datos semiestructurados del perfil (Big Data)

JSON

Índice GIN en PostgreSQL

N

NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.

2

Tabla

estudiante

Descripción Tabla

Subtipo de USUARIO. Estudiante activo matriculado en la institución.

v1.0

#

Campo

Descripción Campo

Tipo Dato - MySQL

Tamaño

Nulo

PK

FK

UK

Nota/Comentarios

1

id_usuario

FK al superíndice usuario

INT

No

S

S

PK y FK a usuario

2

codigo_estudiante

Código estudiantil

VARCHAR(20)

20

3

semestre

Semestre actual (1-10)

SMALLINT

CHECK BETWEEN 1 AND 10

4

id_programa

FK al programa académico

INT

S

FK a programa_academico

N

NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.

3

Tabla

docente

Descripción Tabla

Subtipo de USUARIO. Profesor vinculado a la institución.

v1.0

#

Campo

Descripción Campo

Tipo Dato - MySQL

Tamaño

Nulo

PK

FK

UK

Nota/Comentarios

1

id_usuario

FK al superíndice usuario

INT

No

S

S

PK y FK a usuario

4.2 Script de Creación — MySQL

El script mysql-crea.sql crea la base de datos pascualina y las 30 tablas con ENGINE=InnoDB DEFAULT CHARSET=utf8mb4. Las adaptaciones clave respecto al script de PostgreSQL son:

- Reemplazo de SERIAL por INT AUTO_INCREMENT en todas las PKs autoincrementales.
- Reemplazo de BOOLEAN por TINYINT(1) en los campos leído y leída.
- Reemplazo de TIMESTAMP DEFAULT NOW() por DATETIME DEFAULT CURRENT_TIMESTAMP.
- Uso de DECIMAL(10,2) en lugar de NUMERIC(10,2) para el campo precio — ambos son equivalentes en MySQL.
- Adición de ENGINE=InnoDB en todas las tablas para garantizar soporte de claves foráneas — MyISAM (motor por defecto en versiones antiguas) no soporta FK.
- DEFAULT CHARSET=utf8mb4 para soporte completo de Unicode incluyendo emojis.
- Las restricciones CHECK se soportan desde MySQL 8.0.16 — el script es compatible con MySQL 8.x.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

MySQL Workbench

local x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

pascualina

- Tables
- Views
- Stored Procedures
- Functions

Query 1 20261-bd1-g100-ba-04-E-11-scr

Limit to 1000 rows

```
22 • CREATE DATABASE IF NOT EXISTS pascualina
23     CHARACTER SET utf8mb4
24     COLLATE utf8mb4_unicode_ci;
25
26 • USE pascualina;
27
28 -- =====
29 -- TABLAS INDEPENDIENTES (sin claves foráneas)
30 -- =====
31
32 • CREATE TABLE programa_academico (
33     id_programa INT NOT NULL AUTO_INCREMENT,
34     codigo VARCHAR(20) NOT NULL,
35     nombre VARCHAR(150) NOT NULL,
36     facultad VARCHAR(100) NULL,
37     nivel VARCHAR(50) NULL,
38     CONSTRAINT pk_programa_academico PRIMARY KEY (id_programa),
39     CONSTRAINT uq_programa_codigo UNIQUE (codigo)
40 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
41
42 • CREATE TABLE habilidad (
```

Administration Schemas

Information

No object selected

Output

Action Output

#	Time	Action	Message
1	14:23:26	CREATE DATABASE IF NOT EXISTS pascualina CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci	1 row(s) affected
2	14:23:26	USE pascualina	0 row(s) affected
3	14:23:26	CREATE TABLE programa_academico (id_programa INT NOT NULL AUTO_INCREMENT, codigo VARCHAR(20)...	0 row(s) affected
4	14:23:27	CREATE TABLE habilidad (id_habilidad INT NOT NULL AUTO_INCREMENT, nombre VARCHAR(100) NOT NU...	0 row(s) affected
5	14:23:27	CREATE TABLE interes (id_interes INT NOT NULL AUTO_INCREMENT, nombre VARCHAR(100) NOT NULL, ...	0 row(s) affected
6	14:23:27	CREATE TABLE empresa (id_empresa INT NOT NULL AUTO_INCREMENT, nombre VARCHAR(150) NOT NU...	0 row(s) affected
7	14:23:27	CREATE TABLE usuario (id_usuario INT NOT NULL AUTO_INCREMENT, nombre VARCHAR(100) NULL, a...	0 row(s) affected
8	14:23:27	CREATE TABLE estudiante (id_usuario INT NOT NULL, codigo_estudiante VARCHAR(20) NULL, semestre S...	0 row(s) affected
9	14:23:27	CREATE TABLE docente (id_usuario INT NOT NULL, codigo_docente VARCHAR(20) NULL, departamento VAR...	0 row(s) affected
10	14:23:27	CREATE TABLE empleado (id_usuario INT NOT NULL, codigo_empleado VARCHAR(20) NULL, cargo VARCH...	0 row(s) affected
11	14:23:27	CREATE TABLE egresado (id_usuario INT NOT NULL, anio_egreso SMALLINT NULL, cargo_actual VARCH...	0 row(s) affected
12	14:23:27	CREATE TABLE publico_general (id_usuario INT NOT NULL, organizacion VARCHAR(150) NULL, tpo_vinculo ...	0 row(s) affected
13	14:23:27	CREATE TABLE usuario_habilidad (id_usuario INT NOT NULL, id_habilidad INT NOT NULL, CONSTRAINT pk_usuari...	0 row(s) affected
14	14:23:27	CREATE TABLE usuario_interes (id_usuario INT NOT NULL, id_interes INT NOT NULL, CONSTRAINT pk_usuario_in...	0 row(s) affected
15	14:23:27	CREATE TABLE seguimiento (id_seguidor INT NOT NULL, id_seguido INT NOT NULL, fecha_ingreso DATETI...	0 row(s) affected

Object Info Session

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)



Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

5. Diccionario de Datos y Scripts de MS SQL Server

5.1 Diccionario de Datos Físico — MS SQL Server

El diccionario físico de SQL Server se elaboró en la hoja 'Diccionario Datos - MS SQL Serv' del archivo resultados.xlsx, con los tipos de dato nativos de SQL Server 2019+. Las diferencias más significativas son BIT para booleanos, VARCHAR(MAX) para texto libre, GETDATE() para fecha actual e IDENTITY(1,1) para autoincrementales.

MS SQL Server - Tablas en Detalle									
1	Tabla	usuario	Descripción Tabla	Supertipo central. Atributos comunes de todos los usuarios registrados en Pascualina.					v1.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	Identificador único del usuario	INT IDENTITY(1,1)		No	S			PK autoincremental
2	nombre	Nombre(s) del usuario	VARCHAR(100)	100					
3	apellido	Apellido(s) del usuario	VARCHAR(100)	100					
4	email	Correo electrónico del usuario	VARCHAR(150)	150	No			S	Valor único
5	hash_contraseña	Hash de la contraseña	VARCHAR(255)	255					Nunca texto plano
6	fecha_registro	Fecha y hora de registro	DATETIME						DEFAULT NOW()
7	foto_perfil	URL de la foto de perfil	VARCHAR(255)	255					
8	biografía	Texto de presentación	VARCHAR(MAX)	MAX					
9	estado	Estado de la cuenta	VARCHAR(20)	20					DEFAULT 'activo'; CHECK
10	perfil_extendido	Datos semiestructurados del perfil (Big Data)	NVARCHAR(MAX)						Índice GIN en PostgreSQL
N									
NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.									
2	Tabla	estudiante	Descripción Tabla	Subtipo de USUARIO. Estudiante activo matriculado en la institución.					v1.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	FK al supertipo usuario	INT		No	S	S		FK y FK a usuario
2	codigo_estudiante	Código estudiantil	VARCHAR(20)	20					
3	semestre	Semestre actual (1-10)	SMALLINT						CHECK BETWEEN 1 AND 10
4	id_programa	FK al programa académico	INT				S		FK a programa académico
N									
NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.									
3	Tabla	docente	Descripción Tabla	Subtipo de USUARIO. Profesor vinculado a la institución.					v1.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	FK al supertipo usuario	INT		No	S	S		FK y FK a usuario
2	codigo_docente	Código docente	VARCHAR(20)	20					
3	departamento	Departamento académico	VARCHAR(100)	100					
4	especialidad	Área de especialización	VARCHAR(150)	150					
N									
NOTA: En las columnas PK, FK y UK coloque "S" cuando aplique. En "Nulo" coloque "No" si el campo es obligatorio.									
4	Tabla	empleado	Descripción Tabla	Subtipo de USUARIO. Personal administrativo u operativo.					v1.0
#	Campo	Descripción Campo	Tipo Dato - MS SQL Server	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios
1	id_usuario	FK al supertipo usuario	INT		No	S	S		FK y FK a usuario
2	codigo_empleado	Código de empleado	VARCHAR(20)	20					

5.2 Script de Creación — MS SQL Server

El script sqlserver-crea.sql crea la base de datos y el esquema pascualina, y las 30 tablas con la sintaxis propia de SQL Server. Las adaptaciones más relevantes son:

- Uso de IDENTITY(1,1) en lugar de SERIAL o AUTO_INCREMENT para las PKs autoincrementales.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

- Uso de BIT para los campos booleanos leído y leída — valor 0=false, 1=true.
- Uso de VARCHAR(MAX) en lugar de TEXT — este último está deprecado en SQL Server desde la versión 2005.
- Uso de GETDATE() en lugar de NOW() o CURRENT_TIMESTAMP para los DEFAULT de fecha-hora.
- Separación de sentencias con GO para garantizar la correcta ejecución en lotes (batches) desde SSMS.
- Las relaciones reflexivas (seguimiento) y las cadenas de FK que generarían ciclos usan ON DELETE NO ACTION en lugar de CASCADE — SQL Server no permite cascadas en referencias circulares. La eliminación se gestiona por lógica de aplicación o triggers.
- Las relaciones con múltiples rutas de cascada (como usuario → publicacion y usuario → comentario → publicacion) también usan NO ACTION para evitar errores de múltiple ruta de eliminación.

Explorador de objetos

Conectar ▾

localhost\SQLEXPRESS (SQL Server 17.0.1000 - LAPTOP-89SGH1KL)

Bases de datos

Diagramas de base de datos

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

pascualina.asistencia_evento

pascualina.comentario

pascualina.docente

pascualina.egresado

pascualina.empleado

pascualina.empresa

pascualina.estudiante

pascualina.evento

pascualina.grupo

pascualina.habilidad

pascualina.intercambio

pascualina.interes

pascualina.mensaje

pascualina.miembro_grupo

pascualina.notificacion

pascualina.oferta_laboral

pascualina.oferta_requisito

pascualina.postulacion

pascualina.producto

pascualina.producto_foto

pascualina.programa_academico

pascualina.publicacion

pascualina.publicacion_multimedia

pascualina.publico_general

pascualina.reaccion

pascualina.reporte

pascualina.seguimiento

pascualina.usuario

pascualina.usuario_habilidad

pascualina.usuario_interes

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Administración

Generador de eventos XEvent

20261-bd1-g1...L\Sara (52)

```

49 GO
50
51 CREATE TABLE pascualina.habilidad (
52     id_habilidad INT NOT NULL IDENTITY(1,1),
53     nombre VARCHAR(100) NOT NULL,
54     categoria VARCHAR(100) NULL,
55     CONSTRAINT pk_habilidad PRIMARY KEY (id_habilidad),
56     CONSTRAINT uq_habilidad UNIQUE (nombre)
57 );
58 GO
59
60 CREATE TABLE pascualina.interes (
61     id_interes INT NOT NULL IDENTITY(1,1),
62     nombre VARCHAR(100) NOT NULL,
63     categoria VARCHAR(100) NULL,
64     CONSTRAINT pk_interes PRIMARY KEY (id_interes),
65     CONSTRAINT uq_interes UNIQUE (nombre)
66 );
67 GO
68
69 CREATE TABLE pascualina.empresa (
70     id_empresa INT NOT NULL IDENTITY(1,1),
71     nombre VARCHAR(150) NOT NULL,
72     descripcion VARCHAR(MAX) NULL,
73     sitio_web VARCHAR(255) NULL,
74     contacto VARCHAR(150) NULL,
75     CONSTRAINT pk_empresa PRIMARY KEY (id_empresa)
76 );
77 GO
78
79 -- =====
80 -- SUPERTIPO USUARIO
81 -- =====
82
83 CREATE TABLE pascualina.usuario (

```

100 %

● No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-04-17T23:02:32.6710558-05:00

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6. Análisis de Resultados (Grupo)

6.1 Comparativa de implementaciones DDL

La implementación del modelo físico de la Red Social Estudiantil Pascualina en tres Sistemas Gestores de Bases de Datos distintos — PostgreSQL, MySQL y SQL Server — evidenció de manera concreta que el mismo modelo lógico puede traducirse a DDL de formas significativamente diferentes según el motor destino. Esta experiencia es valiosa porque en entornos profesionales reales es frecuente que un sistema deba ser migrado o desplegado en distintos ambientes (desarrollo local, producción en la nube, legacy corporativo), y la capacidad de adaptar el DDL sin perder la integridad del modelo es una competencia esencial.

SGBD	Licencia	Escalabilidad	Adecuación
PostgreSQL (SGBD 1 — elegido Tarea 3)	Open source / gratuita	Alta — JSONB, GIN, extensiones	Muy alta — ya implementado y validado
MySQL (SGBD 2)	Open source / dual license	Alta — amplio ecosistema	Alta — JSON nativo desde 5.7.8
MS SQL Server (SGBD 3)	Comercial / Express gratuita	Muy alta — enterprise-grade	Alta — SSMS, integración con Azure

Concepto	PostgreSQL	MySQL	SQL Server
Autoincremental	SERIAL	INT AUTO_INCREMENT	INT IDENTITY(1,1)
Booleano	BOOLEAN	TINYINT(1)	BIT
Texto libre	TEXT	TEXT / LONGTEXT	VARCHAR(MAX)
Fecha y hora	TIMESTAMP	DATETIME	DATETIME
Fecha actual	NOW()	CURRENT_TIMESTAMP	GETDATE()
JSON nativo	JSON / JSONB	JSON (desde 5.7.8)	No nativo — NVARCHAR(MAX)
Motor de tablas	No aplica	ENGINE=InnoDB requerido	No aplica
Esquemas	CREATE SCHEMA	USE database	CREATE SCHEMA + USE
FK reflexiva	CASCADE permitido	CASCADE permitido	NO ACTION requerido
Separador lotes	No aplica	No aplica	GO entre batches
Índice JSON	GIN (muy eficiente)	Índice funcional	No nativo para JSON

La tabla anterior sintetiza las diferencias más críticas entre los tres motores. Estas diferencias no son menores: un script PostgreSQL ejecutado directamente en SQL Server fallará en prácticamente todas las tablas por diferencias en tipos de dato, sintaxis de PK y ausencia del separador GO. Esto subraya la importancia de mantener un diccionario de datos físico separado por SGBD, tal como lo exige el enunciado de esta tarea.

6.2 Integridad referencial y acciones referenciales

La implementación de las relaciones foráneas presentó diferencias importantes entre motores. En PostgreSQL y MySQL fue posible usar ON DELETE CASCADE en la mayoría de las relaciones, incluyendo las reflexivas

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

(seguimiento). En SQL Server, las relaciones reflexivas y las cadenas de FK con múltiples rutas de cascada requieren ON DELETE NO ACTION obligatoriamente — el motor rechaza con error las cascadas que podrían generar ciclos o rutas ambiguas de eliminación.

Tabla Padre	Tabla Hija/Pivote	FK	ON DELETE	Justificación
usuario	estudiante/docente/empleado/egresado/publico_general	id_usuario	CASCADE	El subtipo no existe sin el supertipo.
programa_academico	estudiante / egresado	id_programa	SET NULL	El programa puede eliminarse sin perder al usuario.
usuario	seguimiento	id_seguidor / id_seguido	NO ACTION*	*SQL Server: no permite CASCADE en FK reflexivas.
usuario	grupo	id_creador	RESTRICT	No se puede eliminar un usuario con grupos activos.
grupo	miembro_grupo / evento	id_grupo	CASCADE / SET NULL	Miembros se eliminan; eventos persisten sin grupo.
usuario	publicacion / mensaje / reporte	id_usuario	CASCADE	Contenido del usuario desaparece con él.
publicacion	comentario / reaccion / multimedia	id_publicacion	CASCADE	Dependientes directos de la publicación.
empresa	oferta_laboral	id_empresa	RESTRICT	No se elimina empresa con ofertas activas.
producto	producto_foto / intercambio	id_producto	CASCADE / RESTRICT	Fotos en cascada; transacciones protegidas.

Esta restricción de SQL Server no implica pérdida de funcionalidad — significa que la lógica de eliminación debe implementarse en stored procedures o triggers a nivel de aplicación. Es una decisión de diseño del motor que prioriza la explicitéza sobre la conveniencia.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

6.3 Datos semi-estructurados y conexión con Big Data e IoT

La inclusión de campos JSONB en PostgreSQL representa el punto de convergencia entre el modelo relacional tradicional y los paradigmas NoSQL. En el caso de Pascualina, los dos campos JSONB implementados resuelven problemas concretos que el modelo relacional puro no podría atender eficientemente sin comprometer la normalización:

El campo `perfil_extendido` en usuario almacena atributos heterogéneos por tipo de usuario (redes sociales, certificaciones, idiomas, disponibilidad para mentoría). Modelar estos atributos en columnas relacionales requeriría decenas de columnas NULLables o tablas adicionales de atributo-valor que generarían JOINS costosos. JSONB con índice GIN permite consultas como 'usuarios disponibles en tardes interesados en Bases de datos' en una sola operación de índice.

El campo `metricas_evento` en evento captura datos de comportamiento en tiempo real — aforo IoT, temperatura, lecturas NFC para eventos presenciales; viewers, retención, distribución de dispositivos para eventos virtuales. Esta información es altamente variable por tipo de evento y evoluciona con el tiempo (nuevos sensores, nuevas métricas), haciendo impracticable un esquema fijo. En un contexto de producción, herramientas de streaming como Apache Kafka podrían alimentar este campo en tiempo real.

MySQL maneja JSON pero sin el tipo JSONB ni el índice GIN — las consultas sobre campos JSON son más costosas. SQL Server no tiene soporte nativo de JSON como tipo de dato y requiere almacenarlo en VARCHAR(MAX) con funciones auxiliares, perdiendo las ventajas de indexación. Esta diferencia posiciona a PostgreSQL como la opción técnicamente superior para escenarios de Big Data e IoT dentro del contexto de Pascualina.

7. Reflexiones y Conclusiones Individuales

Sara

Implementar el mismo modelo en tres SGBDs distintos me obligó a entender cada motor de manera concreta, no solo en abstracto. Antes de esta tarea, las diferencias entre PostgreSQL, MySQL y SQL Server me parecían superficiales — nombres distintos para lo mismo. Después de escribir los tres scripts de creación, queda claro que las diferencias son profundas y tienen consecuencias reales en diseño, mantenimiento y rendimiento.

La parte que más me hizo reflexionar fue la restricción de SQL Server con las relaciones reflexivas. En PostgreSQL y MySQL, la tabla seguimiento tiene ON DELETE CASCADE en ambas claves foráneas sin ningún problema — si un usuario se elimina, sus relaciones de seguimiento desaparecen automáticamente. En SQL Server, el motor rechaza esta configuración con un error explícito de 'ciclo o múltiples rutas de cascada'. Esto no es un bug ni una limitación arbitraria: es una decisión de diseño que obliga al desarrollador a ser explícito sobre qué sucede cuando se elimina un registro en una relación circular. Tuve que investigar el patrón correcto — NO ACTION en la FK y lógica de eliminación manejada por aplicación — y eso me enseñó más sobre integridad referencial que meses de clases teóricas.

La inclusión de campos JSONB fue la parte conceptualmente más rica de esta tarea. Entender cuándo un modelo relacional puro es insuficiente, y por qué los datos de perfil de usuario o las métricas de eventos justifican un enfoque semi-estructurado, conecta el contenido del curso con tendencias reales del mercado — arquitecturas híbridas SQL+NoSQL que hoy son el estándar en sistemas de escala como redes sociales, plataformas de streaming y sistemas de monitoreo IoT. El hecho de que PostgreSQL soporte JSONB con índices GIN de manera nativa, mientras MySQL y SQL Server tienen soporte limitado o ninguno, es una diferenciación técnica concreta que puedo usar para argumentar la elección de un motor en un contexto profesional real.

En cuanto al trabajo en equipo con Julián, esta tarea requirió una coordinación más cuidadosa que las anteriores porque las decisiones de uno afectan directamente el trabajo del otro: si cambio el nombre de una tabla en el inventario, los tres diccionarios y los tres scripts deben actualizarse en sincronía. Esa interdependencia me hizo más consciente de la importancia de una fuente de verdad única — en este caso el inventario de tablas — y de comunicar los cambios de manera explícita y oportuna.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Julián

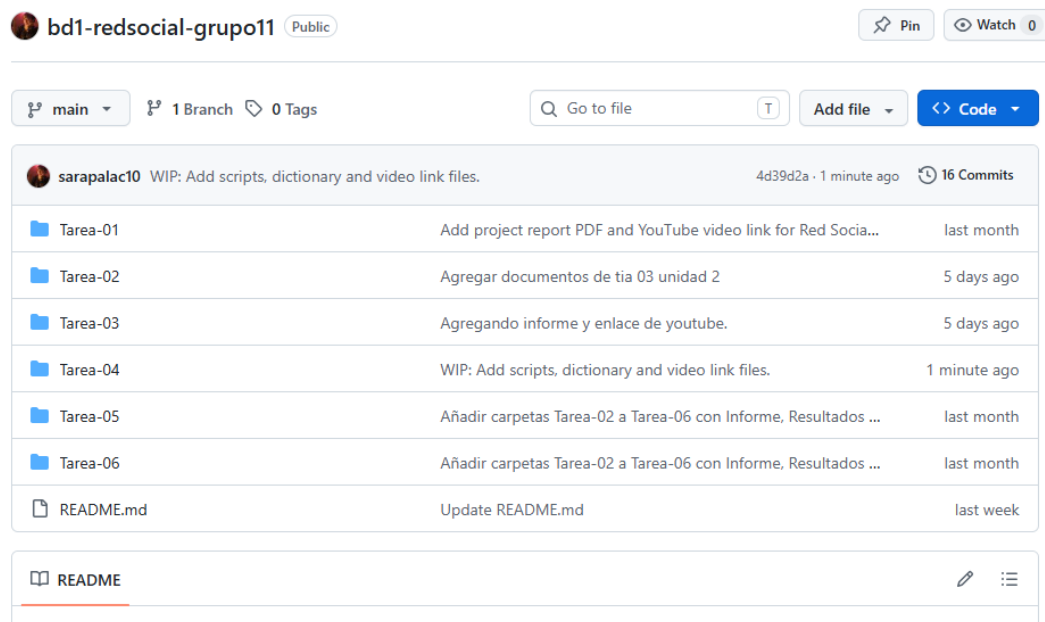
Esta tarea fue la que más claramente me mostró por qué el perfil de un desarrollador de software incluye conocimiento de múltiples SGBDs. En la práctica laboral, es común heredar sistemas legacy en SQL Server mientras el nuevo desarrollo va en PostgreSQL, o tener que migrar una aplicación de MySQL a otro motor por razones de costo o rendimiento. La capacidad de leer y escribir DDL en los tres motores principales, entendiendo sus diferencias y no solo copiando y pegando, es una competencia diferenciadora.

Lo que más me costó de esta tarea fue el script de SQL Server. La sintaxis de `IDENTITY(1,1)`, los bloques `GO`, la ausencia de tipo `BOOLEAN`, el reemplazo de `TEXT` por `VARCHAR(MAX)` — cada una de estas diferencias requirió investigación y prueba. Pero la parte más complicada fue entender el comportamiento de las acciones referenciales en cadena. SQL Server tiene un sistema de detección de ciclos más estricto que PostgreSQL: no solo rechaza las FK reflexivas con `CASCADE`, sino también cualquier conjunto de FKs que genere más de una ruta de eliminación en cascada hacia la misma tabla. Esto afectó varias tablas del modelo — no solo seguimiento — y requirió revisar la arquitectura de eliminación de todo el sistema.

El script de modificación estructural (`postgre-mod-01`) me resultó más fluido que en la Tarea 3 porque ya tenía el contexto de cómo funciona el ciclo de vida de una tabla en PostgreSQL. Esta vez elegí la tabla etiqueta (renombrada a tag) porque el concepto de tags o palabras clave en una red social es genuinamente relevante para el sistema — no es una tabla arbitraria para cumplir el ejercicio. Poder pensar la tabla en su contexto funcional (folksonomy, descubrimiento de contenido, campañas temporales) hizo que las decisiones de diseño (cuándo agregar un `UNIQUE`, por qué crear un índice sobre categoría, por qué eliminar `fecha_fin`) tuvieran una lógica natural en lugar de ser mecánicas.

Finalmente, la conexión entre JSONB y Big Data / IoT fue el aspecto que más impacto tuvo en mi visión del campo. Durante años escuché que 'NoSQL vs SQL' era una elección binaria. Esta tarea me mostró que la realidad es más matizada: PostgreSQL permite tener un modelo relacional robusto y normalizado, y al mismo tiempo incorporar campos semiestructurados JSONB con índices eficientes donde el esquema fijo no es la herramienta correcta

Enlace a repositorio: <https://github.com/sarapalac10/bd1-redsocial-grupo11.git>



Enlace de Youtube: <https://youtu.be/z4foM8pKlyo>

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica de Evaluación

#	Ítem	Peso	Cal.
1	Tabla Comparativa de tipos de Datos	20	
2	Inventario de tablas (anterior tarea)	10	
3.1	Diccionario de Datos Físico (PostgreSQL)	40	
3.2	Scripts de CREACIÓN en PostgreSQL	30	
3.3-3.4	Scripts MODIFICACIÓN PostgreSQL (Estructurados + Semi)	20	
4.1	Diccionario de Datos Físico (MySQL)	40	
4.2	Scripts de CREACIÓN en MySQL	30	
5.1	Diccionario de Datos Físico (MS SQL Server)	40	
5.2	Scripts de CREACIÓN en MS SQL Server	30	
6	Análisis de resultados (en equipo - 300 palabras mínimo)	20	
7	Reflexiones y Conclusiones individuales (300 palabras mínimo)	30	
8	Informe: Estructura, Contenido y Calidad de presentación	40	
9	Video de Sustentación	100	
10	Repositorio GIT	20	
11	Foro de Tareas	30	
NOTA = xx / 500 =		500	